Osciloscopio digital basado en Arduino y Python para visualización de señales analógicas en tiempo real

Emmanuel Morales Loaiza

Facultad de Ingeniería, Instituto Tecnológico Metropolitano

1 Introducción

Los instrumentos de medición electrónica, como los osciloscopios, son herramientas fundamentales en el análisis de señales eléctricas. Sin embargo, su costo puede ser elevado para estudiantes o entornos educativos con recursos limitados.

2 Objetivo general

Desarrollar un sistema de adquisición de datos y visualización de voltaje en tiempo real utilizando Arduino y python.

3 Objetivos específicos

-Implementar la lectura de señales analógicas mediante el ADC del Arduino

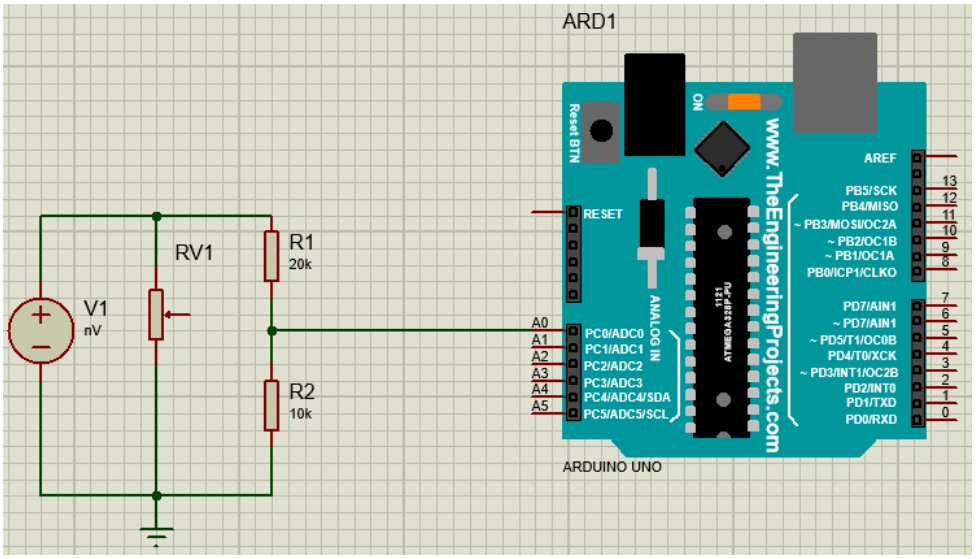
-Transmitir los datos por comunicación serial

-Diseñar una interfaz gráfica para visualizar la señal

4 Marco teórico

Es esencial a tener en cuenta que este sistema se basa en un divisor de voltaje para poder hacer uso correcto de este, dado que un Arduino por si solo puede hacer lecturas máximas de sus pines analógicos solo hasta 5V.

A continuación se presenta el esquema de funcionamiento del sistema.



El voltaje pasa al potenciómetro, de manera que se pueda regular y posteriormente pasar al divisor de voltaje que utiliza resistencias de $20k\Omega$ y $10k\Omega$ para hacer los cálculos más sencillos, de modo que el voltaje recibido por el Arduino en el pin analógico no cause efectos contraproducentes.

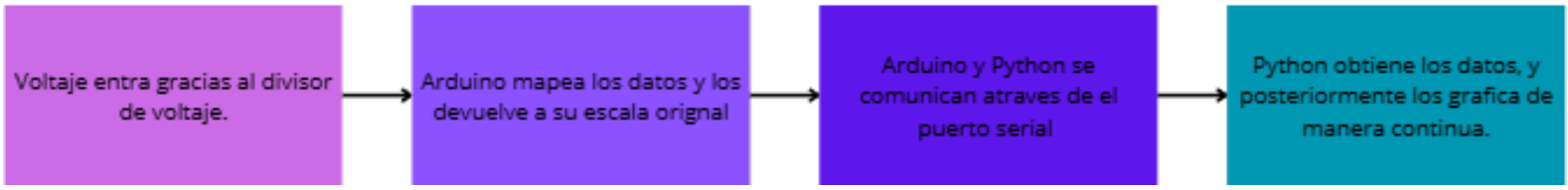
En el código se hace la conversión gracias a la fórmula del divisor de voltaje.

$$V_{out} = V_{in} \cdot \frac{R_2}{R_1 + R_2}$$

Donde V_{in} es el valor analógico medido por el pin, que va de 0 a 1023; la idea de hacer esta operación es devolver el valor a su estado original sin comprometer los componentes electrónicos.

5 Comunicación Arduino/Python

Arduino, es el que me genera los datos y python es el que decide que hacer con ellos; básicamente son dos mundos totalmente diferentes y pensaría uno que son incompatibles entre si y si, es cierto, por eso se utiliza un paso intermedio para comunicarlos a los dos, que se llama comunicación por el puerto serial; por ende se presenta el siguiente esquema.



Entonces, el Arduino envía los datos en forma de Bits, Python los decodifica y lo restante es "carpintería" a nivel de código.

6 Implementación

osciloscopio.ino

```
int lectur = A0;
float R1 = 20000.0;
float R2 = 10000.0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  float data = analogRead(lectur) * (5.0/1023.0);
  float vout = data*((R1 + R2)/R2);
  Serial.println(vout);
  delay(100);
}

```

Figure 1: Primero se hace le código en Arduino y después se sube para que el microcontrolador quede con el ultimo programa cargado.
En este caso lo que se hace es decir que se lea la entrada de voltaje, que inicie el puerto serial, y posteriormente, tomar el dato de el pin, hacer la conversión, y después devolverlo a su factor original.

```python
import pygame as py
import serial
import collections
import tkinter as tk
from tkinter import ttk
import serial.tools.list_ports

root = tk.Tk()
root.title("Seleccionar puerto")

ports = [p.device for p in serial.tools.list_ports.comports()]

choose_port = tk.StringVar()

combo = ttk.Combobox(root,textvariable=choose_port)
combo['values'] = ports
combo.pack(padx=20,pady=20)

def choose():  1 usage

    global selection
    selection = combo.get()
    root.destroy()

button = ttk.Button(root,text='Selecet',command=choose)
button.pack(padx=20,pady=20)
```

Figure 2: Para comunicar esto con python, lo que hacemos primero es importar todas las librerías con las funcionalidades que se necesitan.
Después de todo importado, se empieza el código definiendo la geometría de la ventana en la que se da la opción de seleccionar el puerto serial, se hace una lista con los posibles puertos, se crea una lista desplegable, con los puertos disponibles y se crea una función que al dar clic en el botón de select toma el valor de la lista como el puerto COM

```
root.mainloop()

if selection:
    ser = None
    try:
        ser = serial.Serial(selection, baudrate: 9600, timeout=0.1)
    except Exception as e:
        print(e)

    py.init()

    wd = py.display.set_mode((800, 400))
    font = py.font.Font( name: None, size: 40)

    clk = py.time.Clock()

    status = True

    points = collections.deque([200] * 800, maxlen=800)

    actual_voltage = 0
```

Figure 3: Se inicializa la ventana, y posteriormente se instancia un condicional que pregunta si la selección de el puerto tiene algún valor, si la respuesta es si, sucede lo siguiente, se hace la conexión serial, se inicializara la ventana de pygame, y se crea una lista que almacena los últimos datos obtenidos y elimina los viejos, para que no haya conflicto, se instancia el voltaje en 0.

```python
while status:

    for event in py.event.get():
        if event.type == py.QUIT:
            status = False

    if ser and ser.in_waiting > 0:
        line = ser.readline().decode('utf-8').strip()
        try:
            voltage = float(line)
            actual_voltage = voltage
            y = int(400 - (voltage * 40))
            points.append(y)
        except Exception as e:
            print(e)

    wd.fill((20, 20, 30))

    text = font.render( text: f"voltage: {actual_voltage} V",  antialias: True,  color: (0, 255, 0))
    wd.blit(text,  dest: (10, 10))

    for x in range(0, 800, 50):
        py.draw.line(wd,  color: (60, 60, 60),  start_pos: (x, 0),  end_pos: (x, 400))
    for yl in range(0, 800, 50):
        py.draw.line(wd,  color: (60, 60, 60),  start_pos: (0, yl),  end_pos: (800, yl))
    for i in range(len(points) - 1):
        py.draw.line(wd,  color: (0, 255, 0),  start_pos: (i, points[i]),  end_pos: (i + 1, points[i + 1]),
```

Figure 4: se hace un bucle while que se consultando por el estado de que se instancia anteriormente como True, para que el sistema sea continuo, después se hace un condicional, que pregunta si existe alguna acción, y si esta acción es darle a la X de la parte superior, se cierra la ventana, después se hace un condicional, en el que se dice que si las lineas pasadas por el serialcom son mayores a 0 entonces, se crea una linea, en el que se decodifican los bits pasados, se lee el dato, se calcula la subida y después se almacenan en el arreglo de puntos; si lo anterior no se puede hacer se le dice al sistema que imprima la excepción o error que sucedió.; se crea la interfaz, con el valor de el voltaje en tiempo real y las lineas para simular el osciloscopio

```
        py.display.flip()
        clk.tick(60)

    ser.close()
    py.quit()
else:

    window = tk.Tk()
    window.geometry("600x70")
    Text = tk.Label(window,text="Please review the serial port of conect the Arduino and reboot program.",width=100)
    Text.pack(padx=20,pady=20)
    window.mainloop()
```

Figure 5: finalmente se le dice al sistema que se actualiza con una tasa de 60Hz y que si el sistema se cierra, pues que cierre la ventana.
Ahora, en caso de que en el principio la conexión con el Arduino no fuera posible porque no se encontró el COM, se crea una ventana que dice que revise la conexión y reinicie la aplicación.

7 Limitaciones

Para hacer que el sistema funcione correctamente, se debe tener claro lo siguiente:
-No tener abierto el Serial Com en el ide de Arduino al momento de ejecutar la aplicación, caso contrario el voltaje no va a cambiar y se va a quedar en 0.

-El osciloscopio solo mide corriente directa, de unos máximo 12 voltios, con la configuración de el sistema. Si se quiere aumentar el valor nominal de las resistencias debe ser mas grande y volver a calcular el valor de escala.

8 Resultados obtenidos:

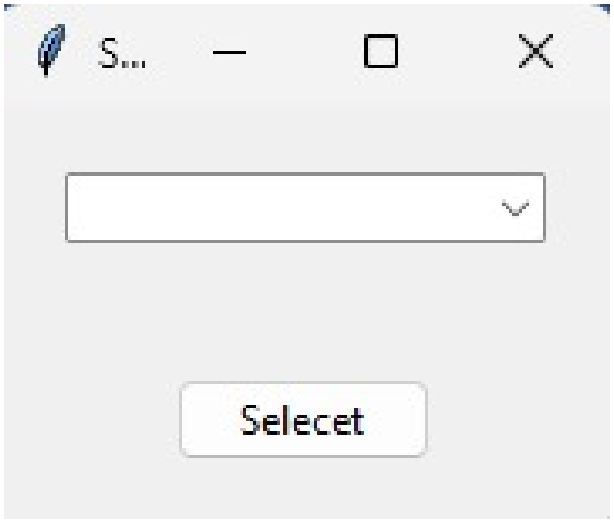


Figure 6: Selector de el puerto serial.

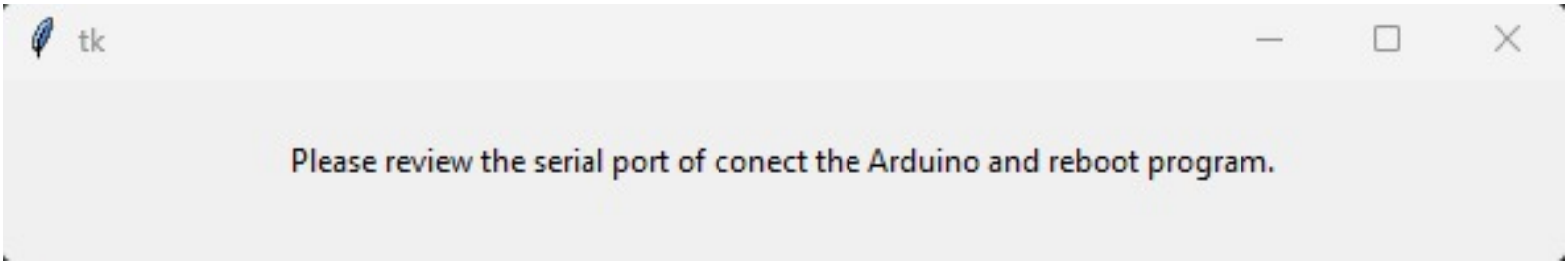


Figure 7: Si no hay puerto, sale el mensaje de error

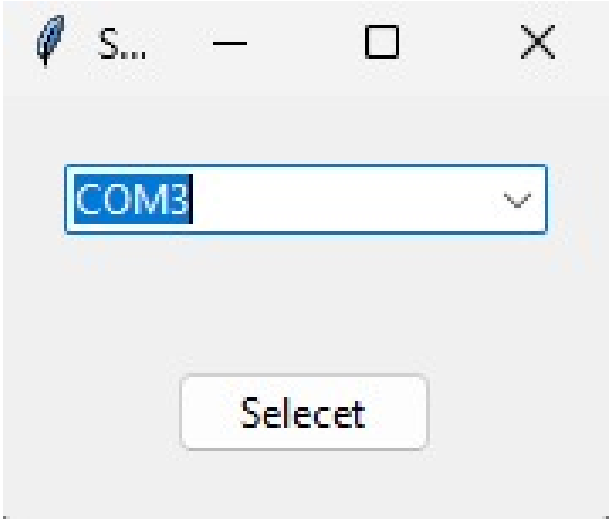


Figure 8: Selector del puerto serial con puerto seleccionado.

Figure 9: Al iniciar correctamente, se debe ver esta pantalla, en la que sale la linea recta, solo se mantiene así hasta que se empiezan a variar los datos.

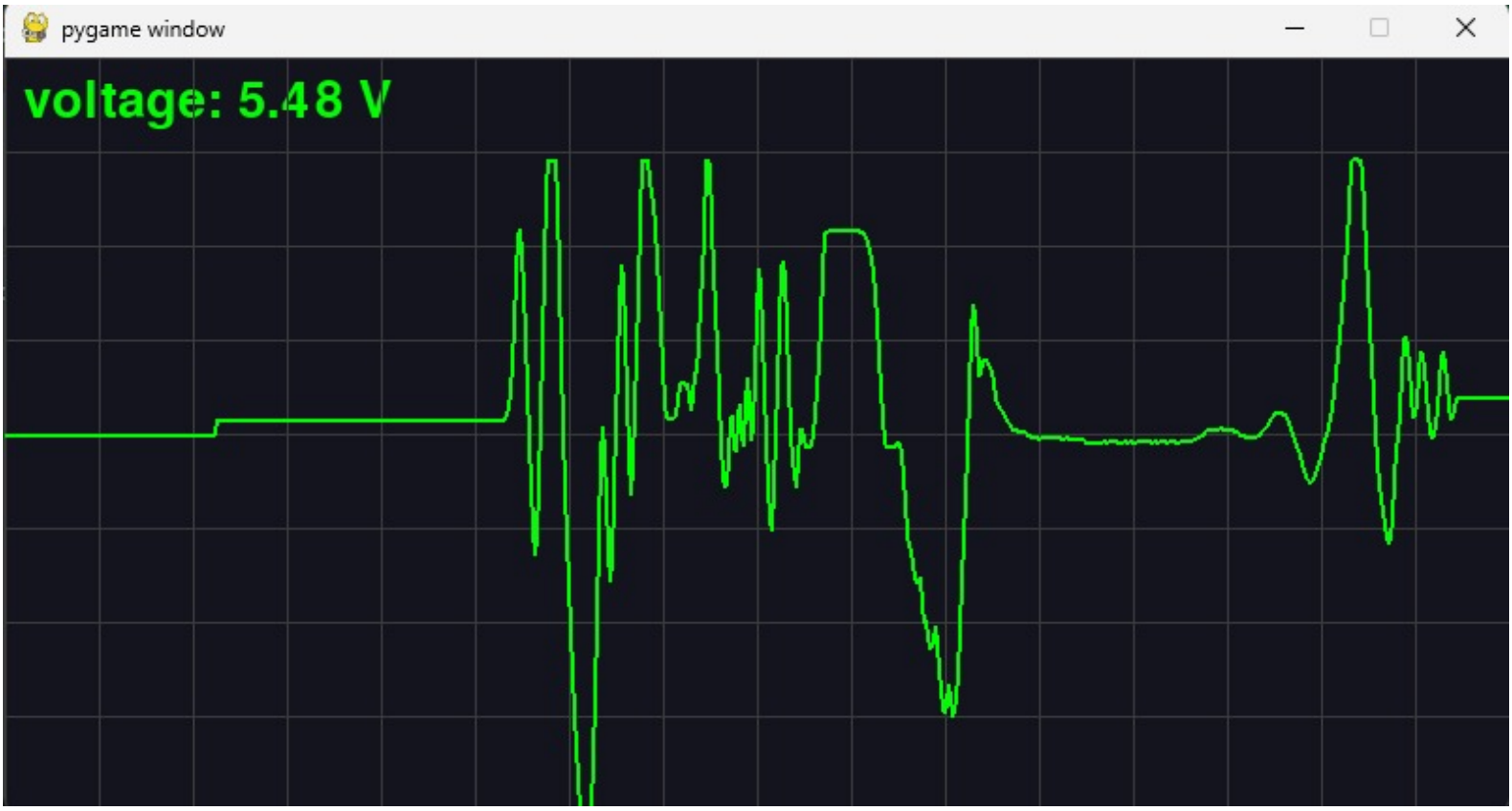


Figure 10: Ejemplo de una variación amplia

Figure 11: Ejemplo de una variación un poco mas controlada

Figure 12: Ejemplo de una variación más estable

9 Conclusiones

-Se logro la creación de un sistema lowcost para osciloscopio simple para DC (Aproximadamente 25000 mil pesos colombianos).

-La grafica es coherente con respecto a los valores medidos (Error de: 0.3V)

10 Trabajo a futuro y posibles mejoras

Como un sistema simple, a pesar de tener sus limitantes, es una herramienta muy buena de trabajo que permite a aquellos que estar iniciando comprender mejor la fenomenología de los sistemas; y así como es simple, permite hacer cambios para la mejora de todo el sistema en si:
-Aumentar la capacidad de el voltaje medido en DC.

-Permitir la medición de señales AC.

-Analizar frecuencias.

-Hacer filtros digitales.

-Hacer almacenamiento de datos.

Finalmente, el código y el archivo ejecutable se encuentra en mi GitHub en la parte de Releases: https://github.com/SienceNova/Proyectos.git